

Theoretische Informatik

Laurin Seeholzer

September 20, 2026

Contents

1	Introduction	3
1.1	Turing Machines	3
1.2	P vs. NP	4
1.3	Polynomial Time Reductions	4
2	Finite Automata	5

1 Introduction

Leibnitz proposed that some time in the future, mathematics will be developed enough to model and solve every possible problem. Then came Russel

Definition 1.1. Russels Paradox: Let S be the set of sets that do not contain themselves.

$$S = \{X \mid X \notin X\}$$

Russels Paradox poses the following Question: Does S contain itself. The answer is paradoxical

$$S \notin S \implies S \in S$$

$$S \in S \implies S \notin S$$

Remark 1.2. Hilbert style deduction allows you to solve Russels paradox by first stating what the axioms and inference rules of the System are.

Theorem 1.3. Goedels incompleteness: There are mathematical statements that are true but can not possibly be proven to be so.

1.1 Turing Machines

Definition 1.4. Turing-Machine (TM): A theoretical machine consisting of an infinity strand of paper, a perfect pen and a set of rules. Every step includes reading current position, modifying it, moving left/right.

Remark 1.5. It can be shown that every algorithm can be modeled as a Turing-Machine and vice-versa.

Theorem 1.6. Turing-Theorem: There are problems that can not be automatically solved using an algorithm/TM.

Proof. Let $w_{i,1}, w_{i,2}, w_{i,3} \dots$ be the output of Turing Machine i for inputs $1, 2, 3 \dots$. We define a new Turing Machine M_x such that $w_{x,j} \neq w_{j,j}$ hence the output of M_x differs from the output of every other Turing machine for at least one input. Observe that this is a valid Turing Machine but it can not possibly be contained in our current list of Turing Machines. Hence, we reach a contradiction: if M_x were in our enumeration at some index x , its output on input x would be $w_{x,x} \neq w_{x,x}$, which is logically impossible. Therefore, our assumption that we can construct such a machine M_x must be false. This implies there is no computable general algorithm capable of determining the output $w_{j,j}$ for all

Turing machines j , proving that there exist uncomputable functions and that the Halting Problem is undecidable. $\square$

Example 1.7. Halting-problem: Does algorithm A terminate?

This problem seems easy at first but we can design programs that for example only terminate once they find an even number n such that there are no primes p, q with $n = p + q$. Solving this (finding such n) would solve the Goldbach-Conjecture which we currently do not know to be true/false.

1.2 P vs. NP

Definition 1.8. P vs. NP: The p/np problem poses the question if all in polynomial time verifiable problems (NP) have a polynomial time solution (P).

Definition 1.9. A problem A in NP is called **NP-Complete** if the existence of an efficient solution for A implies the existence of an efficient solution of all NP problems.

1.3 Polynomial Time Reductions

Definition 1.10. A clique in a graph is a set of vertices such that they are all connected to each other.

Definition 1.11. An independent-Set in a Graph is a set of vertices, such that none of them are connected to each other.

Example 1.12. Assume we know "does G have a clique of size k " is NP-Complete. How can we show that then "Does G have a stable set of size k " is also NP-Complete.

Proof. Observe that inverting a graph (Adding edges where there are none and removing edges where there are) creates a graph G' such that every clique in G is a stable Set in G' and viceversa. This lets us calculate stable sets from cliques and vice versa hence if one problem is NP-Complete both have to be. $\square$

2 Finite Automata

Definition 2.1. Alphabet: finite, non-empty set.

Example 2.2. Examples of Alphabets:

- Boolean alphabet $0, 1$
- latin alphabet $a, b, \dots, z$
- logical alphabet $0, a, x, (,), \dots$

Definition 2.3. Word: A word over an alphabet A is a finite sequence of symbols of A .

Remark 2.4. The length of a word is the number of symbols it uses. There is always an empty word of length zero.

Remark 2.5. Words can be concatenated or reversed to form new words.

$$\begin{aligned} \text{kon}(x, y) &= xy \\ w = a_1 a_2 a_3 &\implies w^R = a_3 a_2 a_1 \end{aligned}$$

Remark 2.6. The i -th iteration of a word $x = ab$ is $x^i = ababab \dots ab$ the word concatenated with itself i times.

Remark 2.7. A word v is a partial-, prefix- or suffix-word of a word w if v is contained in w , w begins with v or ends with v .

Remark 2.8. For a word x and symbol a the expression $|x|_a$ describes the number of appearances of a in x .

Remark 2.9. For an alphabet with an order imposed on its symbols ($a_1 \leq a_2 \leq \dots$) we can define the harmonic order of its words as a lexicographic order where

$$u \leq v \iff (|u| \leq |v|) \vee (|u| = |v| \wedge u = xs_i u', v = xs_j v', s_i \leq s_j)$$